

cap-p2-ivan-lucas-ej2

February 27, 2026

0.1 COMPUTACIÓN DE ALTAS PRESTACIONES - Práctica 2

1 Autores: Lucas Miranda e Iván Domínguez

1.0.1 Ejercicio 2

Lo primero que haremos será implementar nuestra propia versión del algoritmo “Stencil 1D” para posteriormente ejecutarla a diferentes niveles de paralelización (instrucciones SIMD; instrucciones SIMD + paralelización del bucle; paralelización para GPU).

Queremos que la versión de este algoritmo sea completamente *in-place*. Y para evitar problemas más adelante en cuestiones de paralelismo, se ha pensado implementar la versión Red-Black ordering. Consiste en evitar las condiciones de carrera sumando primero las celdas pares (las “rojas”) y después las impares (las “negras”). Tal vez más adelante sea necesario modificar el algoritmo, el que se presenta a continuación no es más que una versión preliminar:

```
[1]: import numpy as np

def stencil_1d(t:np.ndarray, r:int):

    """

    Implementa una versión inicial del famoso algoritmo Stencil 1D. Más
    concretamennte, su versión Red-Black ordering que trata de evitar
    condiciones de carrera (que pueden suponer un problema más adelante).

    t (np.ndarray): array sobre el que se aplicará el algoritmo (inplace).
    r (int): radio de valores tenidos en cuenta.

    """

    # Nos cercioramos de que el radio sea un entero.
    assert isinstance(r, int), "El radio r debe ser un entero."
    # Guardamos la longitud en memoria para más adelante.
    n = len(t)
    # Y guardamos una versión auxiliar de t para no guardar
    # sus elementos de 1 en 1.
    t_old = t.copy()
```

```

# Primero operaremos sobre los pares:
for i in range(0, n, 2):

    # Actualizamos el valor (suma de la vecindad)
    t[i] = np.sum(t_old[(max(0, i - r)):(min(n, i + r + 1))])

# Y después sobre los impares:
for i in range(1, n, 2):
    # Al ser in-place, aquí t[i-1] y t[i+1] ya podrían ser los valores
    ↪ nuevos
    t[i] = np.sum(t_old[(max(0, i - r)):(min(n, i + r + 1))])

return t

```

Un ejemplo de uso sencillo para comprobar la funcionalidad sería el de aplicar `stencil_1d` al array `[1,2,3,4,5]` con radio 2. Podemos saber trivialmente que el resultado es `[6,10,15,14,12]` Comprobémoslo:

```

[2]: t=np.array([1,2,3,4,5])
      stencil_1d(t, 2)
      t

```

```

[2]: array([ 6, 10, 15, 14, 12])

```

Efectivamente, el resultado es correcto y entendemos por tanto que nuestra implementación del algoritmo es correcta.

El objetivo ahora es comparar en términos de rendimiento las distintas formas que tenemos de ejecutar esta función. Como dijimos anteriormente:

- Instrucciones SIMD: compilando la función con el decorador `njit` de `numba`.
- Instrucciones SIMD + paralelización del bucle: compilando la función con el decorador `njit` de `numba` y usando `prange` para paralelizar los bucles.
- Paralelización para GPU: desharemos los bucles y usaremos índices.

Observación: esperamos que, en términos de rendimiento, los dos primeros métodos sean más rápidos. El problema es demasiado trivial, por lo que es esperable que resolverlo con CPU sea más rápido que tan si quiera pasar el problema a la memoria GPU (>200ms).

La primera versión es muy fácil de implementar. Es más, como tal, solo tenemos que añadir el decorador y cambiar el nombre a la función.

```

[3]: from numba import njit

@njit(parallel=False)
def stencil_1d_simd(t:np.ndarray, r:int):

    """

```

Implementa una versión inicial del famoso algoritmo Stencil 1D. Más concretamente, su versión Red-Black ordering que trata de evitar condiciones de carrera (que pueden suponer un problema más adelante).

Con la llamada al decorador @njit aseguramos la compilación del código, así como las instrucciones vectorizadas.

*t (np.ndarray): array sobre el que se aplicará el algoritmo (inplace).
r (int): radio de valores tenidos en cuenta.*

```
"""

# Nos cercioramos de que el radio sea un entero.
assert isinstance(r, int), "El radio r debe ser un entero."
# Guardamos la longitud en memoria para más adelante.
n = len(t)
# Y guardamos una versión auxiliar de t para no guardar
# sus elementos de 1 en 1.
t_old = t.copy()

# Primero operaremos sobre los pares:
for i in range(0, n, 2):

    # Actualizamos el valor (suma de la vecindad)
    t[i] = np.sum(t_old[(max(0, i - r)):(min(n, i + r + 1))])

# Y después sobre los impares:
for i in range(1, n, 2):
    # Al ser in-place, aquí t[i-1] y t[i+1] ya podrían ser los valores
    ↪ nuevos
    t[i] = np.sum(t_old[(max(0, i - r)):(min(n, i + r + 1))])

return t
```

Sin embargo, esta segunda implementación necesita algunos cambios. En concreto, *prange* no acepta saltos de dos en dos. Por tanto, tendremos que reconstruir los índices dentro de los propios bucles. Nada muy complicado:

```
[4]: from numba import prange

@njit(parallel=True)
def stencil_1d_parallel(t:np.ndarray, r:int):
```

```
    """
```

Implementa una versión inicial del famoso algoritmo Stencil 1D. Más concretamente, su versión Red-Black ordering que trata de evitar

condiciones de carrera (que pueden suponer un problema más adelante).

Con la llamada al decorador @njit, no solo aseguramos la compilación del código y las instrucciones vectorizadas, sino que también la paralelización de los bucles (ayudándonos de prange de numba).

```
t (np.ndarray): array sobre el que se aplicará el algoritmo (inplace).
r (int): radio de valores tenidos en cuenta.

"""

# Nos cercioramos de que el radio sea un entero.
assert isinstance(r, int), "El radio r debe ser un entero."
# Guardamos la longitud en memoria para más adelante.
n = len(t)
# Y guardamos una versión auxiliar de t para no guardar
# sus elementos de 1 en 1.
t_old = t.copy()

# Determinamos cuántos elementos pares e impares hay
num_pares = (n + 1) // 2
num_impares = n // 2

# Primero operaremos sobre los pares:
for j in prange(num_pares):
    i = j * 2 # Reconstruimos el índice real (0, 2, 4...)
    t[i] = np.sum(t_old[(max(0, i - r)):(min(n, i + r + 1))])

# Y después sobre los impares:
for j in prange(num_impares):
    i = j * 2 + 1 # Reconstruimos el índice real (1, 3, 5...)
    t[i] = np.sum(t_old[(max(0, i - r)):(min(n, i + r + 1))])

return t
```

Convendría hacer una llamada en frío a las funciones para que se compilen:

```
[5]: # usando el t de unas celdas atrás:
t=np.array([1,2,3,4,5])
stencil_1d_simd(t, 2)
print(t)
t=np.array([1,2,3,4,5])
stencil_1d_parallel(t, 2)
print(t)
```

```
[ 6 10 15 14 12]
[ 6 10 15 14 12]
```

Y ahora implementaremos una tercera versión utilizando Numba CUDA.

Lo primero será ver si tenemos acceso a un dispositivo gráfico. Esta versión está ejecutada desde Kaggle, por lo que para obtener un dispositivo, iremos a settings, accelerator y seleccionaremos el que más nos convenza. **Observación:** los dispositivos gráficos de Kaggle (Nvidia Tesla T4, por ejemplo) fueron creados con la finalidad de ejecutar modelos de aprendizaje automático y programas orientados a la computación de altas prestaciones. Estas gráficas están mucho más alineadas con los propósitos de la asignatura que otras que puedan haber sido diseñadas para correr videojuegos, el tipo de gráficas con las que vienen muchas máquinas hoy en día.

```
[6]: from numba import cuda
      cuda.detect()
```

```
Found 1 CUDA devices
id 0      b'Tesla P100-PCIE-16GB'                                [SUPPORTED]
          Compute Capability: 6.0
          PCI Device ID: 4
          PCI Bus ID: 0
          UUID:
GPU-0e3724fb-5d03-6e9d-5e2a-6ca40d5eba6b
          Watchdog: Disabled
          FP32/FP64 Performance Ratio: 2
Summary:
      1/1 devices are supported
```

```
[6]: True
```

Podemos apreciar que Kaggle nos ofrece 2 gráficas Nvidia Tesla T4 gratuitamente. Aunque solo 30 horas semanales. Seleccionaremos solo 1 de momento por motivos de sencillez.

```
[7]: cuda.select_device(0)
```

```
[7]: <weakproxy at 0x7c40f4658900 to Device at 0x7c40f41cd550>
```

Ahora implementaremos la versión para una gráfica del algoritmo Stencil 1D. Es importante tener en cuenta que tenemos que desprendernos de los bucles. Cada hilo lanzado, ejecutará una instancia de la función que programaremos, por lo que tendremos que asociar un índice para cada hilo. Una vez asegurado esto, tendremos que cerciorar también que los índices están dentro de rango (como mucho, la longitud del array).

Para controlar que la función se ejecuta por partes (primero pares, luego impares), añadiremos un parámetro *phase*. De tal forma, solo se lanzarán las 2 versiones de la función, una para los pares, otra para los impares. El resultado final se pasará de nuevo a memoria principal para poder pasarlo por pantalla.

Para ejemplificar, ya no podemos usar el array [1,2,4,5] porque ni si quiera merece la pena pasarlo a memoria para ejecutarlo con la gráfica. Lo lógico sería probarlo con un array de, por lo menos, 1 millón de elementos. Posteriormente evaluaremos si merece la pena con un número tan bajo de elementos.

```
[8]: @cuda.jit
      def stencil_1d_cuda(t:np.ndarray, r:int, phase:bool):
```

```

"""

Implementa una versión inicial del famoso algoritmo Stencil 1D. Más
concretamente, su versión Red-Black ordering que trata de evitar
condiciones de carrera (que pueden suponer un problema más adelante).

Ahora no ya no hay bucles. La idea ahora es asignar a cada hilo un índice
a modificar.

    t (np.ndarray): array sobre el que se aplicará el algoritmo (inplace).
    r (int): radio de valores tenidos en cuenta.

"""

# Calculamos el índice global del hilo
i = cuda.grid(1)
n = t.shape[0]

# Comprobamos que el hilo esté dentro de los límites y en la fase correcta
if i < n and (i % 2 == phase):
    suma = 0.0
    # Cada hilo hace su propia suma local
    for j in range(max(0, i - r), min(n, i + r + 1)):
        suma += t[j]

    cuda.syncthreads()
    t[i] = suma

```

Ejecutamos:

```

[9]: t=np.random.randn(10**6)
    d_t=cuda.to_device(t)
    n = t.shape[0]
    threads_per_block = 256
    blocks_per_grid = int(np.ceil(n / threads_per_block))
    r=2
    stencil_1d_cuda[blocks_per_grid, threads_per_block](d_t, r, 0)
    stencil_1d_cuda[blocks_per_grid, threads_per_block](d_t, r, 1)
    d_t.copy_to_host()

```

```

[9]: array([-0.03712117,  3.84795776,  2.4291965 , ...,  1.73225315,
           0.75695262, -0.11527004])

```

Podemos ver que, efectivamente, somos capaces de ejecutar esta versión del Stencil 1D. Ahora podemos hacer una comparativa (en tiempo) entre las 3 implementaciones probando con arrays de 100.000 a 1.000.000 en pasos de 100.000.

Nota: también podemos ver como el tamaño del bloque o fijamos en función del tamaño del array. Creemos que esto es lo óptimo para tener un número de hilos mínimo para la longitud de nuestro

problema. Es más, si los tamaños de los arrays fueran potencias de dos, podríamos crear un número de hilos exactamente igual al número de elementos del array, asociando un hilo a cada elemento.

Mediremos los tiempos con la clase *Timer* de la práctica anterior:

```
[10]: # Medición de tiempos

import time

class Timer:

    def __enter__(self):
        self.start = time.time()
        return self

    def __exit__(self, *args):
        self.end = time.time()
        self.interval = self.end - self.start
```

Y guardaremos un tiempo para cada tamaño y método:

```
[11]: # Medimos los tiempos de ejecución de ambas funciones de multiplicaciones de
      ↪matrices en Python

# Podemos reciclar casi todo el código de antes:

tamaño = []
tiempo_simd = []
tiempo_prange = []
tiempo_cuda = []

n_iter = 10

for i in range(10**6, 10**7, 10**6):

    # generamos un array arbitrario de tamaño i
    # muestrean una normal(0,1)
    array = np.random.randn(i)
    # guardamos un tal tamaño:
    tamaño.append(i)
    # fijamos un valor r=10 arbitrario
    radius=10

    # - - - - Stencil 1D SIMD - - - -

    # 1. Creamos un array para almacenar 10 tiempos:
    results_simd = 0
```

```

for _ in range(n_iter):

    # 2. Medimos el tiempo y lo guardamos en el array:

    with Timer() as k:
        stencil_1d_simd(array, radius)

    result = k.interval
    results_simd += result

# 3. Calculamos la media de tiempos

results_simd /= n_iter

# 4. Lo añadimos a los tiempos de slow
tiempo_simd.append(results_simd)

# - - - - Stencil 1D Parallel - - - -

# 1. Creamos un array para almacenar 10 tiempos:
results_prange = 0

for _ in range(n_iter):

    # 2. Medimos el tiempo y lo guardamos en el array:

    with Timer() as k:
        stencil_1d_parallel(array, radius)

    result = k.interval
    results_prange += result

# 3. Calculamos la media de tiempos

results_prange /= n_iter

# 4. Lo añadimos a los tiempos de slow
tiempo_prange.append(results_prange)

# - - - - Stencil 1D CUDA - - - -

# 1. Creamos un array para almacenar 10 tiempos:
results_cuda = 0

for _ in range(n_iter):

    # 2. Medimos el tiempo y lo guardamos en el array:

```



```

with Timer() as k:
    d_t=cuda.to_device(t)
    n = t.shape[0]
    threads_per_block = 512
    blocks_per_grid = int(np.ceil(n / threads_per_block))
    r=2
    stencil_1d_cuda[blocks_per_grid, threads_per_block](d_t, r, 0)
    stencil_1d_cuda[blocks_per_grid, threads_per_block](d_t, r, 1)
    d_t.copy_to_host()

    result = k.interval
    results_cuda += result

# 3. Calculamos la media de tiempos

results_cuda /= n_iter

# 4. Lo añadimos a los tiempos de slow
tiempo_cuda.append(results_cuda)

```

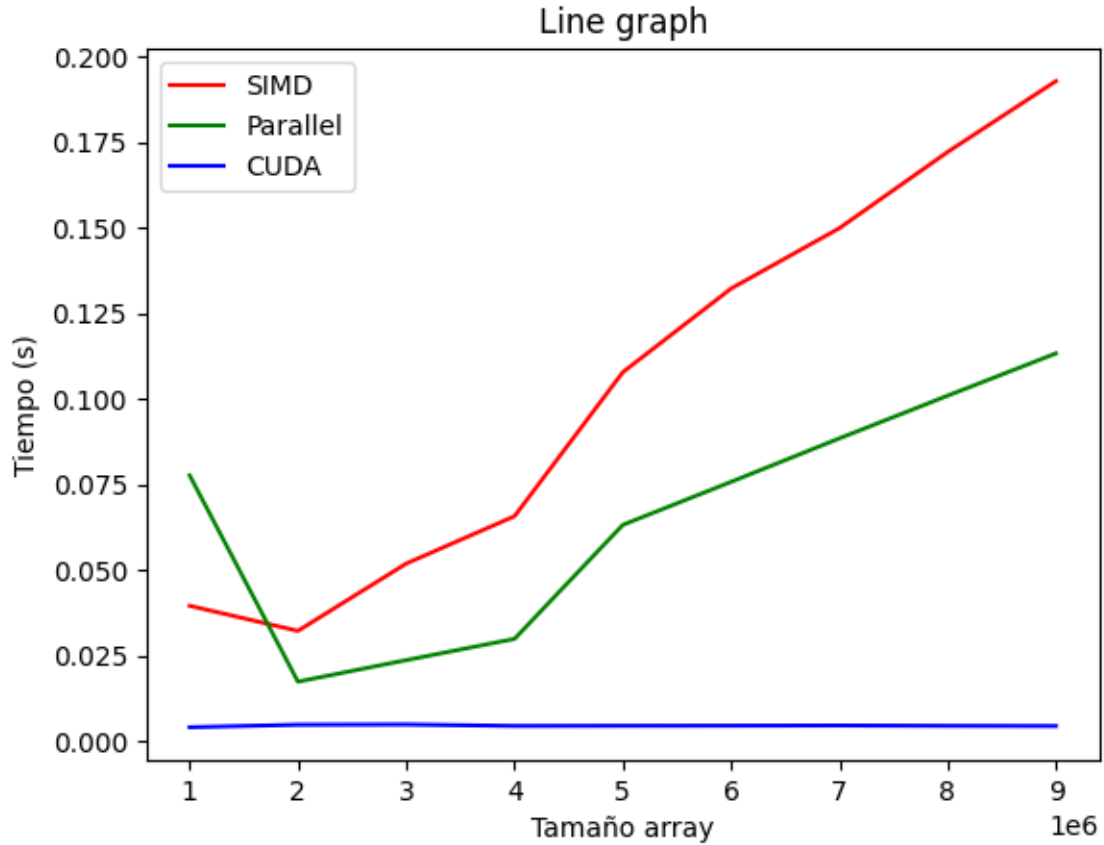
Y graficamos para ver los resultados:

```

[12]: import matplotlib.pyplot as plt

plt.figure()
plt.title("Line graph")
plt.xlabel("Tamaño array")
plt.ylabel("Tiempo (s)")
plt.plot(tamaño, tiempo_simd, color="red", label="SIMD")
plt.plot(tamaño, tiempo_prange, color="green", label="Parallel")
plt.plot(tamaño, tiempo_cuda, color="blue", label="CUDA")
plt.legend(loc="upper left")
plt.show()

```



Es muy gracioso ver como este problema sigue siendo despreciable para la gráfica, mientras que para la CPU ya empieza a ser costoso para arrays de 5 millones de elementos.

Como se explicaba anteriormente, el tamaño de bloque se ha creado en función del número de elementos del array. Esto es, fijando un número arbitrario de hilos por bloque -512, por ejemplo-, se crea un número mínimo de bloques de tal forma que cada elemento del array tenga asociado un hilo (habrá -pocos- hilos que no tengan un elemento del array asociado). Matemáticamente esto es la función techo del tamaño del array entre el numero de hilos por bloque:

$$n_{\text{bloques}} = \left\lceil \frac{\text{length}(\text{array})}{n_{\text{hilos}}^{\text{bloque}}} \right\rceil$$

Creemos que esto es lo óptimo, pues de esta forma no se crean más hilos de los necesarios. Incluso podríamos reducir el número de hilos para que, al aumentar el número de bloques, el porcentaje de hilos sin tareas del último de los bloques sea menos (por puras cuestiones de probabilidad). Para este problema, sin embargo, el efecto es despreciable.